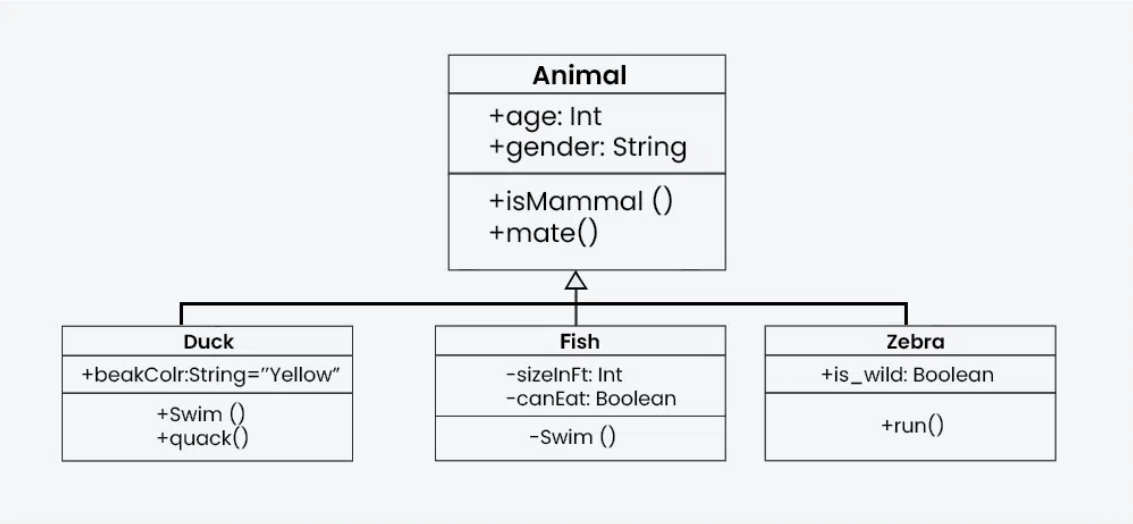
Search...
Sign In
Non Functional Life Cycle Design Patterns UML Diagrams Sys
Sign In

UML Class Diagram

Last Updated : 29 Aug, 2025

A UML class diagram visually represents the structure of a system by showing its classes, attributes, methods, and the relationships between them.

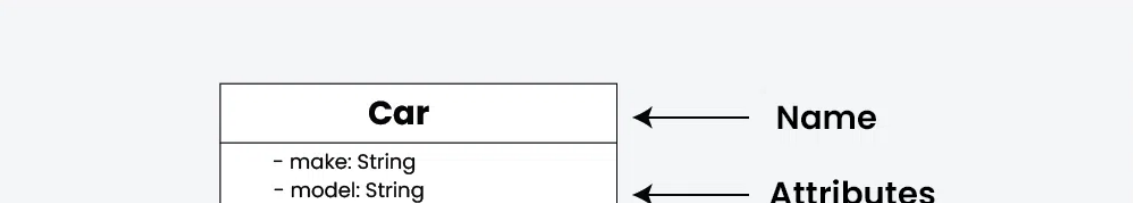
1. Helps everyone involved in a project—like developers and designers—understand how the system is organized and how its components interact.
2. Helps to communicate and document the structure of the software.

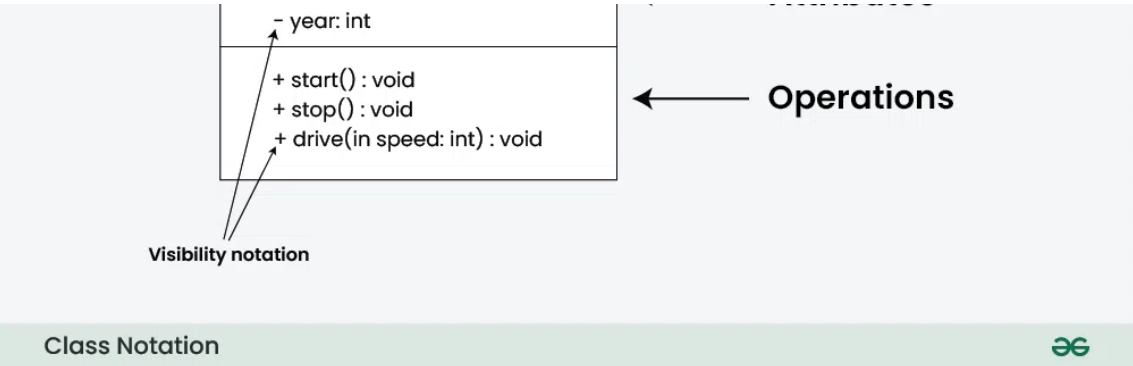


Class Diagram example

UML Class Notation

Classes are depicted as boxes, each containing three compartments for the class name, attributes, and methods.





1. **Class Name:**
   - The name of the class is typically written in the top compartment of the class box and is centered and bold.

2. **Attributes:**
   - Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.

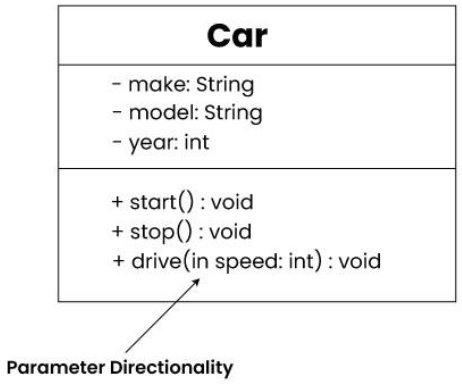
3. **Methods:**
   - Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.

4. **Visibility Notation:**
   - Visibility notations indicate the access level of attributes and methods. Common visibility notations include:
     - + for public (visible to all classes)
     - - for private (visible only within the class)

- # for protected (visible to subclasses)
- ~ for package or default visibility (visible to classes in the same package)

Parameter Directionality

- In class diagrams, parameter directionality refers to the indication of the flow of information between classes through method parameters.
- It helps to specify whether a parameter is an input, an output, or both. This information is crucial for understanding how data is passed between objects during method calls.



class notation with parameter directionality

There are three main parameter directionality notations used in class diagrams:

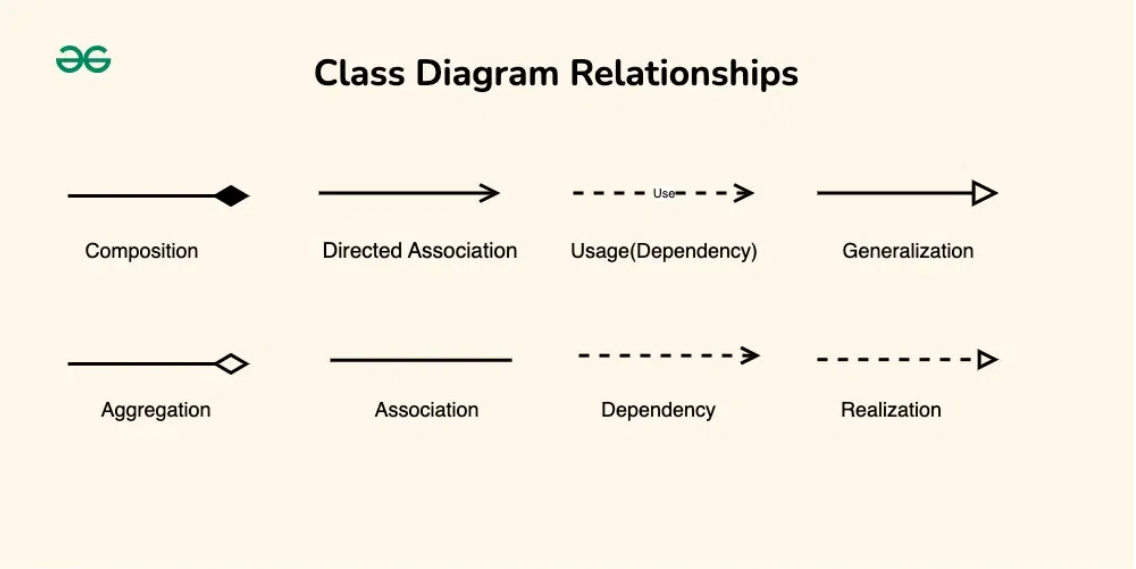
- **In (Input):**
  - An input parameter is a parameter passed from the calling object (client) to the called object (server) during a method invocation.
  - It is represented by an arrow pointing towards the

receiving class (the class that owns the method).

- **Out (Output):**
  - An output parameter is a parameter passed from the called object (server) back to the calling object (client) after the method execution.
  - It is represented by an arrow pointing away from the receiving class.
- **InOut (Input and Output):**
  - An InOut parameter serves as both input and output. It carries information from the calling object to the called object and vice versa.
  - It is represented by an arrow pointing towards and away from the receiving class.

Relationships between classes

In class diagrams, relationships between classes describe how classes are connected or interact with each other within a system. Here are some common types of relationships in class diagrams:



1. Association

An association represents a bi-directional relationship between two classes. It indicates that instances of one class are connected to instances of another class. Associations are typically depicted as a solid line connecting the classes, with optional arrows indicating the direction of the relationship.

2. Directed Association

A directed association in a UML class diagram represents a relationship between two classes where the association has a direction, indicating that one class is associated with another in a specific way.

3. Aggregation

Aggregation is a specialized form of association that represents a "whole-part" relationship. It denotes a stronger relationship where one class (the whole) contains or is composed of another class (the part). Aggregation is represented by a diamond shape on the side of the whole class. In this kind of relationship, the child class can exist independently of its parent class.

4. Composition

Composition is a stronger form of aggregation, indicating a more significant ownership or dependency relationship. In composition, the part class cannot exist independently of the whole class. Composition is represented by a filled diamond shape on the side of the whole class.

5. Generalization(Inheritance)

Inheritance represents an "is-a" relationship between classes, where one class (the subclass or child) inherits the properties and behaviors of another class (the superclass or parent). Inheritance is depicted by a solid line with a closed, hollow arrowhead pointing from the subclass to the superclass.

6. Realization (Interface Implementation)

Realization indicates that a class implements the features of an interface. It is often used in cases where a class realizes the operations defined by an interface. Realization is depicted by a dashed line with an open arrowhead pointing from the implementing class to the interface.

7. Dependency Relationship

A dependency exists between two classes when one class relies on another, but the relationship is not as strong as association or inheritance. It represents a more loosely coupled connection between classes.

8. Usage(Dependency) Relationship

A usage dependency relationship in a UML class diagram indicates that one class (the client) utilizes or depends on another class (the supplier) to perform certain tasks or access certain functionality. The client class relies on the services provided by the supplier class but does not own or create instances of it.

- In UML class diagrams, usage dependencies are typically

represented by a dashed arrowed line pointing from the client class to the supplier class.
- The arrow indicates the direction of the dependency, showing that the client class depends on the services provided by the supplier class.

Purpose of Class Diagrams

The main purpose of using class diagrams is:

- This is the only UML that can appropriately depict various aspects of the OOPs concept.
- Proper design and analysis of applications can be faster and efficient.
- It is the base for deployment and component diagram.
- It incorporates forward and reverse engineering.

Benefits of Class Diagrams

Below are the benefits of class diagrams:

- Class diagrams represent the system's classes, attributes, methods, and relationships, providing a clear view of its architecture.
- They shows various relationships between classes, such as associations and inheritance, helping stakeholders understand component connectivity.
- Class diagrams serve as a visual tool for communication among team members and stakeholders, bridging gaps between technical and non-technical audiences.
- They guide developers in coding by illustrating the design, ensuring consistency between the design and actual implementation.
- Many development tools allow for code generation from